VERDANT — Personal Finance, Intentionally Planned

Comprehensive Technical Architecture, Methodology, Implementation Details & Future Improvements

---

1. Executive & Academic Project Summary

Academic Project Title: AI-Based Personalised Savings and Financial Planning System
Product Name: VERDANT — Personal Finance, Intentionally Planned
Course Context: Professional Training I (PT1) Project Review
System Objective: Build a presentation-ready, fully functional full-stack web application that synthesizes an individual's income, expenses, spending patterns, and multi-horizon financial goals into a personalised savings plan. The core philosophy is **mathematical integrity**: rule-based budget constraints govern all allocations, preventing AI hallucinations while an LLM provides contextual qualitative synthesis.

2. Technology Stack & Tools Used

Layer / Component	Technology Used	Rationale & Architectural Purpose
Frontend Framework	React 19, TypeScript, Vite	Modern reactive component architecture with high type safety and sub-second HMR.
Styling & Aesthetics	Tailwind CSS, Google Fonts	'The Living Ledger' editorial aesthetic (Instrument Serif, Plus Jakarta Sans, JetBrains Mono).
Data Visualisation	Recharts	Interactive SVG charts for cash-flow waterfall bars, category distributions, and 12-month wealth roadmap.
Iconography	Lucide React	Consistent, minimalist technical icons for transactions, drawer triggers, and notifications.
Backend Server	Python 3.12, Flask, Flask-CORS	Lightweight, robust WSGI REST API facilitating rapid financial math and database orchestration.
Persistence & DB	SQLite (verdant.db)	Embedded relational database with foreign key support; zero config for student laptop demos.
AI / LLM Integration	Hybrid LLM API + Deterministic Fallback	OpenAI GPT-4o-mini API integration with fallback to rule-based editorial wealth synthesis engine.
Verification & QA	Python Unittest, Vite Production Build	Unit testing of waterfall allocations, surplus bounds, and static TypeScript bundle checks.

3. Implementation Architecture — The Five Core Modules

Module 1: User Profile & Financial Overview

- Implemented user account persistence storing monthly take-home income, fixed obligations, variable commitments, existing liquid savings, and debt schedules.
- Built an academic review demo profile (Arjun Verma): ■50,000 income, ■15,000 fixed, ■10,000 variable, ■3,000 debt obligations, and ■40,000 existing savings.
- Dynamically computes **Net Monthly Surplus = ■22,000** (44% savings rate) and 1.6 months of emergency runway.

Module 2: Expense Tracking & Categorisation

- Refined double-entry style ledger table tracking amount, date, category, description, and payment method.
- Categorised across 10 sectors (Housing, Food, Utilities, Transport, Healthcare, Entertainment, Shopping, Debt, Education, Other).
- Classifies expenses into Essential Needs (50% benchmark) vs Discretionary Wants (30% benchmark) to detect expenditure leakage.
- Integrated full CRUD operations with live category aggregation updating the dashboard instantly.

Module 3: Financial Goal Management

- Tracks multi-horizon goals (Emergency Funds, Higher Education, Travel, Electronics, Debt Clearance).
- Dynamically calculates remaining financial gap, completion %, target horizon in months, and required monthly velocity.
- Built an interactive contribution logger with celebratory notifications upon reaching 100% funding.
- **Overcommitment Detection:** Flags unrealistic milestones if total monthly demand exceeds the verified surplus ceiling.

Module 4: AI-Assisted Strategic Savings Plan Generator (Core Feature)

- Features an engaging 7-stage analytical generation pipeline showcasing real-time data processing.
- **Mathematical Guarantee:** Allocations are bounded such that Sum(Allocated_i) ≤ Net Monthly Surplus. No double counting of debt.
- Simulates a 12-month forward cumulative savings roadmap detailing exact monthly balance progressions per goal.
- Generates discretionary spending trim recommendations and a concrete 30-day tactical execution checklist.
- Contextual editorial commentary powered by OpenAI GPT-4o-mini or the deterministic wealth synthesis engine.

Module 5: Progress Monitoring & Plan Adjustment

- Revision audit system storing previous plan states in SQLite with active flag management.
- **Plan Adjustment Diff:** Compares changes between revisions, quantifying surplus deltas (e.g. -■2,000), milestone delays, and allocation shifts.
- Persistent in-app notifications drawer tracking budget thresholds, milestone alerts, and large outflow warnings.

4. Mathematical Formulation & Algorithmic Design

1. Monthly Net Surplus Formulation:
Surplus = Monthly_Income - (Fixed_Expenses + Variable_Expenses) - Debt_Obligations
Ensures debt payments are explicitly accounted for and not duplicated if already included in fixed expenses.

2. Goal Priority & Allocation Weighting:
Score_i = Priority_Weight_i * (10 / (Months_Remaining_i + 1))
High Priority = 3x (Emergency Funds receive +2 extra emergency weight), Medium = 2x, Low = 1x. Allocations are capped at each goal's ideal monthly requirement to prevent overfunding while starving secondary goals.

3. 50/30/20 Classical Wealth Benchmark:
Needs = 0.50 * Income | Wants = 0.30 * Income | Savings Target = 0.20 * Income.

5. Future Scope & Potential Improvements

1. Automated Bank Feed Integration (Account Aggregator / Plaid / Setu):

Currently, transactions and baseline amounts are user-logged or seeded. Integrating standard Account Aggregator (AA) APIs under RBI guidelines would enable automated bank statement ingestion and real-time expense reconciliation without manual entry.

2. Inflation & Dynamic Compound Yield Projections:

The current 12-month simulation assumes constant currency value and zero or conservative yields. Incorporating inflation adjustments (e.g. 6% CPI) alongside risk-adjusted returns (Fixed Deposits, Debt Funds, Equity Mutual Funds) would deliver higher-fidelity long-term wealth trajectories.

3. Machine Learning Categorisation Engine (Local NLP / Embeddings):

Replace rule-based category matching with a fine-tuned lightweight local NLP model (e.g. FastText or MiniLM) to categorize cryptic merchant descriptions (e.g., 'UPI-ZEPTO-DELHI-023') with high precision.

4. Multi-User Authentication & JWT Security:

While basic secure password hashing (scrypt) is present, the app is currently optimized for seamless student demonstration with a 1-click demo account. Adding JWT bearer tokens, OAuth2 (Google/GitHub), and email verification would elevate it to commercial SaaS readiness.

5. Push Notifications & Weekly WhatsApp/Telegram Digests:

Extend the current in-app notification drawer with web push notifications or a scheduled Telegram/WhatsApp bot to alert users of weekly spending velocity before budgets are breached.